



TTB COLA Label Verification System - Prototype

This repository contains the offline, LAN-based prototype for the TTB COLA Automated Label Verification system. This application uses local Artificial Intelligence (AI) to automatically cross-reference submitted alcohol beverage labels against simulated application paperwork, verifying strict adherence to Federal Government Warning compliance and ABV limits.



Quick Start & Deployment

Because this application runs entirely locally to protect data and bypass firewall restrictions, a one-time environment setup is required on the host machine.

Prerequisites (One-Time Setup)

1. **Python:** Ensure Python (3.11 or 3.12) is installed on your computer. (*Critical: Check the "Add python.exe to PATH" box during installation*).
2. **Ollama:** Download and install the local AI engine from ollama.com. Ensure the application is running (indicated by the alpaca icon in your system tray).

Note: Python and Ollama *must* be installed for the application to launch properly.

How to Launch

1. Extract/Unzip the `TTB_COLA_Prototype` folder to your Desktop.
2. Open the folder and double-click the `setup_and_run.bat` file.
3. A terminal window will open, automatically building a secure virtual environment, downloading the AI vision model (`minicpm-v`), and launching the Agent Dashboard in your web browser.

How to Use the Prototype

1. **Submit:** Use the drag-and-drop portal on the dashboard to upload test files (TIFF, JPEG, PNG up to 1.5MB; or PDFs/DOCs up to 750KB).
2. **Process:** The system will automatically route valid files to the background AI worker.
3. **Review:** The AI cross-references the data, securely scrubs obvious business names (replacing them with a generic Business Registration Number or BRN for anonymity), and generates a compliance report.
4. **Track:** Agents can view the "Complete Master Queue" or switch to the "Action Required: Flagged Applications" tab to solely focus on non-compliant submissions.



Architecture & Technology Stack

The Core Framework: We will stick to Python for the backend logic (`ttb_backend_worker.py`) and Streamlit for the frontend UI (`ttb_agent_interface.py`). It is incredibly straightforward,

requires minimal coding, and is perfectly suited for building clean, big-button interfaces that are highly accessible for users of all technical backgrounds.

Data Handling: You can use JSON to structure the complex TTB rule permutations (e.g., if beverage == "wine", check for X; if beverage == "spirit", check for Y). SQL (like a lightweight SQLite database) is perfect for storing the output records, handling the "processing vs. complete" status queries, and managing the linked company names over time.

The AI Engine: To meet your strict offline requirements, we will use Ollama, an open-source tool that lets you run powerful multimodal AI models locally on your own hardware.



Choices and Reasoning

- **Python:** Python is a malleable language that interfaces with emerging and outdated technologies. It bridges abstractions and enables future migrations and integrations that run more smoothly.
- **Ollama:** Ollama allows for the Application to run without network access. Meaning Data inputs and outputs can function seamlessly, making the application a practical tool for agents instead of a daily frustration. While Ollama may not be approved, it is an open source that justifies a digital twin with similar capability.
- **LAN over Cloud:** Data will be required to be screened and submitted through cyber security and FSO protocols. Thus while the submission of Data is a fundamental question - most of that data will be previously gathered and secured. The objective is to parse the information and have compliance files organized so that 90% of the account work is done and Agents can focus on the review phase, as well as other tasks. Cloud based applications would require network access, creating domain based headaches in the long run.
- **Data Processing Speed:** While the user interface is extremely simple, the backend is designed to process 300 - 500 applications, with a compliance rating in under an hour. This means more productivity, in a fraction of the time, with a major removal of workload from the Agents. This would allow Agents across the board to focus on higher priority tasks.
- **Maleability in an uncertain technological future:** The combination of Python, Ollama as Local OCR/LLM, and a LAN based operation network is what defines the adaptability of the application. With simple downloads/installation that pass security's approval, this prototype aims to set a template for the paperwork department, agency-wide, as well as federally.